

DS 310 — Homework 2

Build & Query the Insighta Database

Grading: 77 points are graded automatically on Gradescope; the Part 4 written questions are graded by hand (4 points each, 12 points). Assignment total: 89 points.

Overview

In this assignment you build the Insighta research-platform database from raw CSV files, load it with Python, and then query it. The notebook self-checks your work, but the official grade is produced on Gradescope.

- **Part 1 — Create the schema.** Write the CREATE TABLE statements for all five tables.
- **Part 2 — Load the data.** Run the provided loader cell that fills your tables from the CSV files — nothing to write, not graded.
- **Part 3 — Query the database.** Answer 11 query questions.
- **Part 4 — Constraints & NULLs.** Short written questions (graded by hand, 4 points each — 12 points total).

Getting started

Open `YourName_DS310-HW2.ipynb` and keep the provided `data/` folder (five CSV files) in the same directory. Run the **Setup** cell first — it creates a fresh `insighta.db`, enables foreign keys (`PRAGMA foreign_keys = ON`), and defines the `run()` helper. The notebook is safe to re-run from top to bottom at any time.

Schema reference

```
User(user_id, email, password, role)
UserPhone(user_id, phone)
Study(study_id, researcher_id, title, study_type, status,
      duration_min, reward_amount, published_at)
ScreeningQuestion(question_id, study_id, question_text, order_num)
Application(application_id, study_id, participant_id, submitted_at, status)
```

Attribute	Possible values
User.role	'researcher', 'participant', 'both'
Study.status	'active', 'paused', 'closed', 'draft'
Study.study_type	'survey', 'interview'
Application.status	'completed', 'pending', 'approved', 'withdrawn'

A blank field in a CSV means **NULL** (for example, draft studies have an empty `published_at`).

Part 1 — Create the schema (20 points)

Write `CREATE TABLE` statements that recreate the schema in `insighta.db`. Declare primary keys, NOT NULL, UNIQUE, composite keys, and foreign keys as described.

Q1.1 — User and UserPhone (6 points)

- User: `user_id` PRIMARY KEY; `email` NOT NULL; plus `password`, `role`.
- UserPhone: composite PRIMARY KEY (`user_id`, `phone`); FOREIGN KEY `user_id` → User(`user_id`).

Q1.2 — Study (4 points)

- `study_id` PRIMARY KEY; `researcher_id` FOREIGN KEY → User(`user_id`); `published_at` may be NULL (drafts); plus `title`, `study_type`, `status`, `duration_min`, `reward_amount`.

Q1.3 — Application (6 points)

- `application_id` PRIMARY KEY; UNIQUE (`study_id`, `participant_id`) (one application per participant per study); FOREIGN KEYS `study_id` → Study(`study_id`) and `participant_id` → User(`user_id`); plus `submitted_at`, `status`.

Q1.4 — ScreeningQuestion (4 points)

- `question_id` PRIMARY KEY; `study_id` FOREIGN KEY → Study(`study_id`); plus `question_text`, `order_num`.

Part 2 — Load the data (provided — not graded)

The `data/` folder has one CSV per table. The loader code is provided in the notebook — read it, then just run the cell. It inserts each CSV's rows with a **parameterized** `INSERT`, then calls **commit**.

Key ideas explained in the notebook: `executemany` runs one `INSERT` per row safely via ? placeholders; `commit` saves the transaction. The loader also skips the header row; treats an empty field as `NULL`, and loads **parent tables before child tables** because foreign keys are enabled.

Part 3 — Query the database (57 points)

Write each query as a SQL string in the given variable and run it. Each query is auto-graded; row comparisons are order-insensitive except where a sort is required.

Q1 — Filter and sort (5 points)

List the `title` and `reward_amount` of every active study whose reward is greater than \$10, sorted by `reward_amount` from highest to lowest.

Return columns: `title`, `reward_amount`

Q2 — Pattern matching (4 points)

List the `email` of every user whose address is a BU address (ends in `@bu.edu`).

Return columns: `email`

Q3 — Join with a filter (5 points)

List the `title` of every study that has received at least one `completed` application. Each title should appear only once.

Return columns: `title (distinct)`

Q4 — Subquery / set membership (5 points)

List the `email` of every user who has never submitted an application. Hint: a `NOT IN` subquery works well.

Return columns: `email`

Q5 — Outer join + aggregate (6 points)

For every study, show its `title` and the total number of applications it has received, including studies with zero applications. Sort by the count, highest first. Hint: `LEFT OUTER JOIN + GROUP BY`; mind `COUNT(*)` vs `COUNT(column)`.

Return columns: `title, num_apps`

Q6 — Group filtering (6 points)

List the `title` of every study whose applications are all `completed` — the study has at least one application and none in any other status.

Return columns: `title`

Q7 — Aggregate in a subquery (5 points)

Show the `title` and `reward_amount` of the active study with the highest reward. Use a subquery — do not use `ORDER BY ... LIMIT`.

Return columns: `title, reward_amount`

Q8 — Counting per group, including zeros (5 points)

For every researcher (identified by `email`), show how many studies they have posted, including researchers who have posted none.

Return columns: `email, num_studies`

Q9 — HAVING (5 points)

List the `email` of every participant who has applied to more than one distinct study.

Return columns: `email`

Q10 — Grouped average with HAVING (5 points)

For each `study_type`, compute the average reward among active studies, and return only the study types whose average exceeds \$9.

Return columns: `study_type, avg_reward`

Q11 — Filtered outer join (6 points)

For every study, show its `title` and the number of `completed` applications it has received, including studies with zero. Think carefully about where the `status = 'completed'` condition must go.

Return columns: `title`, `completed_count`

Part 4 — Constraints & NULLs (written, graded by hand — 4 points each, 12 points)

Answer in the Markdown cells of the notebook. Foreign keys are enabled (`PRAGMA foreign_keys = ON`), so reason about the constraints as actually enforced.

Q4.1 — Allow or reject? (4 points)

For each operation, state `ALLOW` or `REJECT` and give a one-sentence reason.

```
A. INSERT INTO Application VALUES (9999, 202, 16, '2025-06-01', 'pending')
B. INSERT INTO Application VALUES (9998, 202, 16, '2025-06-01', 'pending')
C. INSERT INTO Application VALUES (9997, 999, 16, '2025-06-01', 'pending')
D. DELETE FROM Study WHERE study_id = 201
```

(In A and B the same participant applies to the same study twice; in C study 999 does not exist; D removes a study that has applications.)

Q4.2 — NULLs (4 points)

Studies 217 and 218 are drafts with `published_at = NULL`.

- **A.** Will `SELECT title FROM Study WHERE published_at != '2025-01-10'` return studies 217 or 218? Explain why or why not.
- **B.** Write a query returning all studies that have not yet been published.

Q4.3 — WHERE vs ON (4 points)

A classmate counts non-withdrawn applications per study with `SELECT study_id, COUNT(*) FROM Application WHERE status != 'withdrawn' GROUP BY study_id`, which omits studies that have zero non-withdrawn applications.

- **A.** Explain (i) why a `LEFT OUTER JOIN` from `Study` is needed, and (ii) why the `status != 'withdrawn'` condition must go in the `ON` clause rather than the `WHERE` clause.
- **B.** Write the corrected query: every study with its count of non-withdrawn applications, including zeros.

Q5 — Use of AI (0 points)

Answer the multiple choice question about your use of AI. This question is worth zero points, and your answers do not affect your grades on this or any homework.

Submitting

Run all cells top to bottom (**Kernel → Restart & Run All**), make sure every check passes, then upload `YourName_DS310-HW2.ipynb` to Gradescope. The Part 4 written questions are graded by hand (4 points each, 12 points total).